2020 International Conference on Unmanned Aircraft Systems (ICUAS)
Athens, Greece. September 1-4, 2020

Real-time Motion Planning of Curvature Continuous Trajectories for Urban UAV Operations in Wind

Jay Patrikar[1], Vishal Dugar[1], Vaibhav Arcot[1] and Sebastian Scherer[1]

***Abstract*— A key challenge in enabling autonomous Unmanned Aerial Vehicles (UAVs) to operate in cluttered urban environments is to plan collision-free, smooth, dynamically feasible trajectories between two locations with the wind in real-time. This paper presents a novel path planning strategy using sampling-based planning that uses a two-point boundary value problem (BVP) to connect states in the presence of wind. Unlike most approaches that use a curvature discontinuous solution, the proposed BVP is formulated as a nonlinear constrained optimization problem with curvature and curvature-rate continuous profile to generate smoother trajectories. To achieve real-time performance, our method uses surrogate solutions from a pre-calculated library while solving the planning problem and then runs a repair routine to generate the final trajectory. To validate the feasibility of the offline-online strategy, simulation results on a 3D model of an actual city block with a realistic wind-field are presented. Results with a trochoid-based BVP solver are also presented for comparison. For the given simulation scenario, we could demonstrate a 93% success rate for the algorithm in finding a valid trajectory.**

I. INTRODUCTION

With advancements in technology, the use of drones or unmanned aerial vehicles (UAVs) in dense cityscapes for various operations like surveillance [1], package deliveries [2] and environmental monitoring [3] is all but inevitable. While these systems may offer several advantages, there are a few critical challenges that still need to be addressed to realize these systems in the real world.

Modern cityscapes provide a unique set of challenges for unmanned aerial vehicles. The presence of tall buildings on both sides of the streets creates a canyon-like situation inside urban areas. Generating feasible trajectories within these urban canyons is challenging because a) urban canyons are often densely packed with narrow passageways between adjacent buildings b) urban canyons affect the flow of air, and this leads to complicated and often dangerous wind patterns that may adversely affect UAV operations [4]. Both problems need to be explicitly addressed to generate feasible trajectories that ensure reliable and safe aerial operations.

One possible way to address this issue is to perform wind-agnostic path planning but design a controller that is capable of handling wind disturbances [5] [6]. While these may be intuitive solutions, the systems may produce a large cross-track error if the wind on the planned route leads to actuator saturation. Recent works [7] have tried to address these issues by incorporating actuator saturation in the path following algorithms. One recent approach [8] uses a strategy that switches to a provably-safe controller if the vehicle violates a pre-calculated tracking error bound. This bound captures all possible deviations due to external disturbances like the wind but may lead to a conservative solution. Another approach [9] uses an estimate of the wind and the corresponding drag forces to generate trajectories given a set of waypoints. All of these approaches assume a path planning algorithm for waypoint generation. This work explores the possibility of explicitly including wind in the path planning phase, thereby reducing the burden on the lower level controller. The current approach derives its focus from two broad areas: path planning in vector fields and kinodynamic path planning.

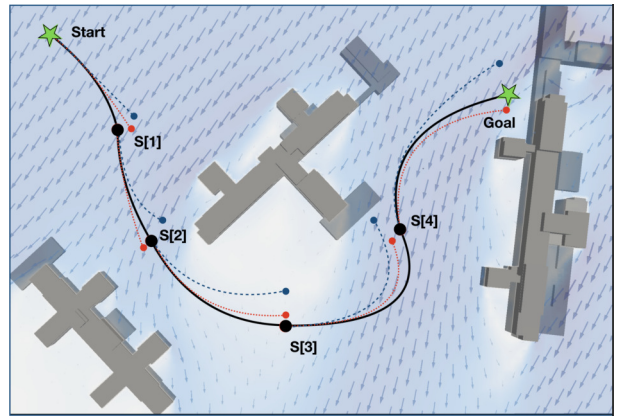


Fig. 1: Figure shows a representative path planning scenario where blue lines indicate the BVP surrogate solutions chosen from the precomputed library, red indicates the wind-corrected solutions used for sampling-based planning algorithm (BIT*) and black lines represent the repaired final trajectory.

Path planning in a vector field like ocean currents or wind is an established problem in robotics. Subramani et al. [10] used level-set PDEs to find a time-optimal path in ocean current but ignore explicit vehicle dynamics as underwater operations are often at lower speeds. There exists a large body of work [11], [12] for calculating energy-optimal paths for autonomous dynamic soaring, but the considered arena is often high altitude obstacle-free airspace. There is a recent interest in solving for drone trajectories in low altitude cluttered urban environments [13]–[16]. However, none of the methods take into account wind and dynamic feasibility.

In the context of kinodynamic path planning for mobile

*This work is supported by Department of Energy Grant DE-EE0008463
[1]Authors are associated with the Robotics Institute, School of Computer Science at the Carnegie Mellon University , Pittsburgh, PA, USA jpatrika@andrew.cmu.edu, vishaldugar12@gmail.com, yvarcot@gmail.com, basti@andrew.cmu.edu

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

robots, recent popular approaches use a probabilistically complete sampling-based planner like BIT* [17] or RRT* [18] that provide asymptotic optimality guarantees. All of these planners need to solve a two-point boundary value problem (BVP) to connect pairs of states with dynamically feasible trajectories. Solving BVPs is often too slow for real-time performance. Methods like neighborhood classification [19], Sequential Quadratic Programming (SQP) formulation [20] , bang-bang control based trajectory generation procedure [21] or resolution-complete online-offline trajectory library pre-computation [22] are used to achieve a speed-up in the planning process but none of them take into account wind. One approach is to avoid solving the BVP by using a forward propagation model [23] but execution times are arguably too slow for real-time.

A. Our Approach

In this paper, we present a novel real-time method to solve wind-aware curvature-smooth two-point boundary value problems to connect states in a sampling-based BIT* global planner. The planner assumes a known map and a known spatially varying but temporally static wind field. We present a BVP formulation that contains segments of higher-order polynomial spirals that can possess as many degrees of freedom as necessary to meet any number of constraints. For the offline phase, wind-agnostic solutions to pre-determined BVPs are used to populate a trajectory primitives library. These library primitives serve as surrogate solutions when the planner requests BVP solutions in the online phase. The requested BVP solutions are modified, assuming locally uniform wind. Once a BIT* solution is generated using these approximate primitives, we repair the path online using the actual nonlinear optimization BVP routine. As the repairs are only carried out on the final path, the number of actual BVPs solved is equal to the number of distinct edges in the final path. The final path is a wind-aware, smooth, collision-free sequence of states between a start and goal location. The approach is detailed in Figure 2

The main contributions of this paper are as follow:

- We formulate a novel wind-aware two-point boundary value problem that produces curvature, and curvature-rate constrained trajectories using piece-wise continuous curvature polynomials.
- We present an offline-online strategy using the surrogate-repair paradigm that uses a pre-computed trajectory library as surrogate solutions to provide real-time performance from a sampling-based planner.

II. Problem Definition

We define the UAV state space $\mathbb{C} = \mathbb{R}^3 \times \mathbb{S}^1$ where $\mathbf{x} \in \mathbb{C}$ is the state of the robot. If we define (x, y, z) as the position coordinates of the robot in inertial space and θ as the heading angle, then $\mathbf{x} = (x, y, z, \theta)^T$ is the vehicle state vector. Throughout the paper we assume a zero side-slip condition. Wind is assumed to be known and spatially-variant but temporally static in the horizontal x-y dimension with the vertical z component assumed to be zero. We further define $\mathbb{C}_{free}$ as the obstacle free state space of the robot. For this work, we use the vehicle kinematics model [24] defined using trajectory distance variable $s \in [0, S_f]$ as follows:

$$x(s) = \int_0^s (cos\theta(s) + \frac{w_x}{V_a})ds \tag{1a}$$

$$y(s) = \int_0^s (sin\theta(s) + \frac{w_y}{V_a})ds \tag{1b}$$

$$z(s) = \int_0^s g(s)ds \tag{1c}$$

$$\theta(s) = \int_0^s \kappa(s)ds \tag{1d}$$

$$\tag{1e}$$

where V_a is the airspeed, w_x and w_y represent the x and y components of the wind vector in the inertial frame respectively[1], κ is the curvature and g is the glideslope. The input space is $\mathbf{u}(s) = (g(s), \kappa(s))^T$. Let $\sigma(s) = (\mathbf{x}(s), \mathbf{u}(s), S_f)^T$ define a path parameterized with $s \in [0, S_f]$. The choice of this non-holonomic system representation is used as it is consistent with many of the current UAV systems.

We may now formally define the problem. Given a start location $\mathbf{x}_s \in \mathbb{C}_{free}$ and a goal location $\mathbf{x}_g \in \mathbb{C}_{free}$, find the optimal path,

$$\begin{aligned} &\min_{\sigma(\cdot)} \quad S_f \\ &s.t. \quad \forall s \in [0, S_f] \\ &\mathbf{x}(s) \in \mathbb{C}_{free}, \;\; \mathbf{x}(0) = \mathbf{x}_s, \;\; \mathbf{x}(S_f) = \mathbf{x}_g \\ &|g(s)| \leq g_{max}, \;\; |\kappa(s)| \leq \kappa_{max}, \;\; |\dot{\kappa}(s)| \leq \dot{\kappa}_{max} \end{aligned} \tag{2}$$

III. Wind-Aware Boundary Value Problem

This section details our approach to formulate a BVP that gives a smooth, dynamically feasible wind-aware solution to exactly connect any states within $\mathbb{C}_{free}$. Traditionally, trochoids have been a popular candidate as a BVP solution for many of the wind-based path planners [25]–[27], because they intuitively represent the transformation due to wind. However, trochoids are discontinuous in curvature space which invalidates their use in our setup. We can still use the same idea and extend trochoids by using polynomials to represent the trochoidal curvature. Using a piece-wise continuous construction of these polynomials, it is possible to get curvature and curvature-rate limited paths that satisfy the boundary conditions. While the global planner is designed to deal with spatially nonuniform wind, for any two-point query we assume a local uniform wind profile by averaging the wind states between the two points. This approximation holds if the sampled states are spatially close assuming that the wind doesn't change drastically over relatively smaller distances (approx. 50m in our setup). The aforementioned sub-problem of finding a smooth trajectory between two states ignores any collision checks, as the global planner, BIT* in our case, handles these. Additionally, the only restriction on z-dimension is the maximum glideslope, and this is taken into consideration in the full planner. The BVP

[1]Here both w_x and w_y are functions of $\mathbf{x}$ and we use locally uniform values when solving the equations.

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

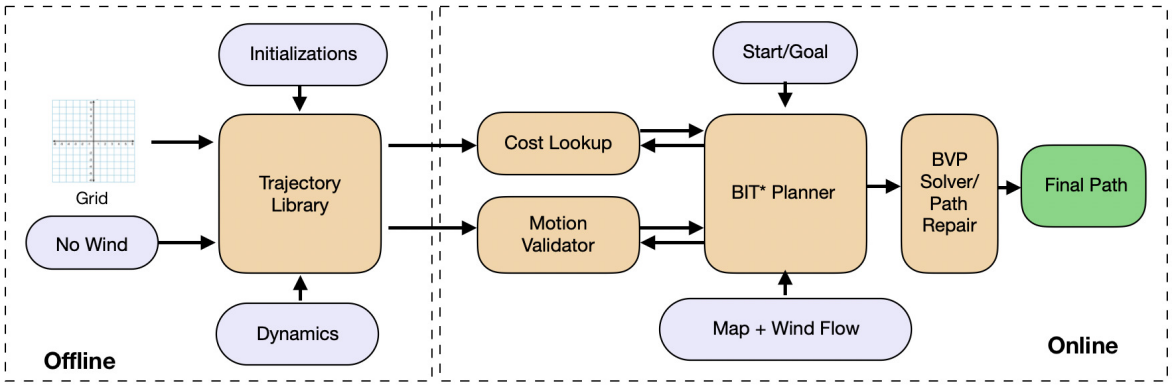


Fig. 2: Overall Approach: Offline, we generate a trajectory library of precomputed wind-agnostic BVP solutions on a predefined grid. Online, we use the trajectory library to provide wind-aware surrogate solutions to perform real-time planning. Only the surrogate solutions that are part of the final path are repaired to provide smooth collision-free wind-aware path.

solution is generated in a SE2 space with $\tilde{\mathbf{x}} = (x, y, \theta)^T$ with κ as control input. This solution is then superimposed on a linear z-profile to connect the states in the actual $\mathbb{C}$ space.

A. BVP Formulation

Given an initial state $\tilde{\mathbf{x}}_0 = (x_0, y_0, \theta_0)^T$, a final state $\tilde{\mathbf{x}}_f = (x_f, y_f, \theta_f)^T$ and a uniform wind field $(w_x, w_y)^T$ between them, find a path $\Phi(s) = (\tilde{\mathbf{x}}(s), \kappa(s), sf)$ parameterized with $s \in [0, sf]$, where sf is path length such that it gives a $\Phi^*(\cdot)$

$$\begin{aligned} &\min_{\Phi(\cdot)} \quad sf \\ &s.t.\ \forall\ 0 \le s \le sf \\ &\kappa(s) \le \kappa_{max}, \quad \kappa'(s) \le \kappa'_{max}, \quad \kappa(0) = 0, \quad \kappa(sf) = 0 \\ &\tilde{\mathbf{x}}(0) = \tilde{\mathbf{x}}_{\mathbf{0}}, \qquad \tilde{\mathbf{x}}(sf) = \tilde{\mathbf{x}}_{\mathbf{f}} \end{aligned} \tag{3}$$

In order to solve this nonlinear optimization problem, we build on previous work [28] that uses polynomial spirals and extend it to generate three piece-wise continuous spirals of lengths sf_1, sf_2 and sf_3 to solve the problem. This formulation is based on Dubins formulation which uses a set of 3 curves with a bang-straight-bang strategy to connect two points [29] optimally. Such strategies have been shown to give optimal paths under the assumption that the distance between points is large enough. Thus the path Φ in Problem 3 can now be formulated as finding a continuous piece-wise path $\Phi(\cdot)$ of length $sf = sf_1 + sf_2 + sf_3$,

$$\Phi(s) = \begin{cases} \phi_1(s) & 0 \le s < sf_1 \\ \phi_2(s) & sf_1 \le s < sf_1 + sf_2 \\ \phi_3(s) & sf_1 + sf_2 \le s \le sf \end{cases} \tag{4}$$

The ith path where $i = \{1, 2, 3\}$ is defined as

$$\phi_i(s) = (\tilde{\mathbf{x}}(s), \kappa_i(s), sf_i)$$

where

$$\kappa_i(s) = a_i + b_i s + c_i s^2 + d_i s^3 + e_i s^4 \tag{5}$$

Keeping the curvature of the middle segment constant and simplifying using conditions in Eq. 3, we get

$$\kappa_1(s) = b_1 s + c_1 s^2 + d_1 s^3 + e_1 s^4 \tag{6a}$$
$$\kappa_2(s) = \kappa_1(sf_1) \tag{6b}$$
$$\kappa_3(s) = \kappa_2(sf_2) + b_3 s + c_3 s^2 + d_3 s^3 + e_3 s^4 \tag{6c}$$

These formulations give us the following set of 11 variables for the nonlinear optimization problem.

$$\mathbf{p} = [b_1, c_1, d_1, e_1, b_3, c_3, d_3, e_3, sf_1, sf_2, sf_3]$$

B. Nonlinear Optimization

The nonlinear optimization formulation is given as

$$\text{minimize:} \quad J(\mathbf{p}) = sf_1 + sf_2 + sf_3 \tag{7a}$$

such that,

$$||\Delta\mathbb{L}|| \le \varepsilon \tag{7b}$$
$$\text{for} \quad 0 \le s < sf_1$$
$$|\kappa_1(s)| \le \kappa_{max} \quad |\dot{\kappa}_1(s)| \le \dot{\kappa}_{max} \tag{7c}$$
$$\text{for} \quad sf_1 \le s < sf_1 + sf_2$$
$$|\kappa_2(s)| \le \kappa_{max} \quad |\dot{\kappa}_2(s)| \le \dot{\kappa}_{max} \tag{7d}$$
$$\text{for} \quad sf_1 + sf_2 \le s \le sf$$
$$|\kappa_3(s)| \le \kappa_{max} \quad |\dot{\kappa}_3(s)| \le \dot{\kappa}_{max} \tag{7e}$$

where,

$$\Delta\mathbb{L} = \begin{bmatrix} x(sf) - x_f \\ y(sf) - y_f \\ L(\theta(sf) - \theta_f) \\ L^2 \kappa(sf) \end{bmatrix} \tag{8}$$

and ε is the error tolerance. The scaling factor L is used to weigh the angle and curvature error against positional error

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

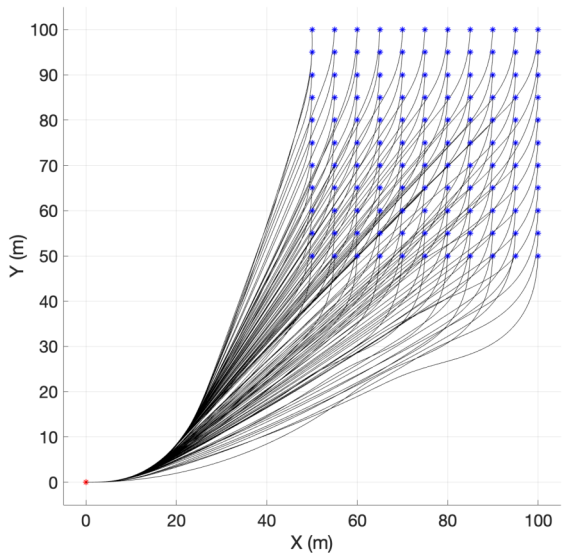


Fig. 3: Representation of a section of the trajectory library with a 90° terminal angle

[28]. The terminal values are calculated using Equations 9.

$$x(sf) = \sum_{i=\{1,2,3\}} x_i(sf_i) + \frac{w_x}{V_a} \tag{9a}$$

$$y(sf) = \sum_{i=\{1,2,3\}} y_i(sf_i) + \frac{w_y}{V_a} \tag{9b}$$

$$\theta(sf) = \theta_3(sf_3) \tag{9c}$$

$$\kappa(sf) = \kappa_3(sf_3) \tag{9d}$$

where for i ={1,2,3},

$$\begin{aligned} x_i(sf_i) &= \int_0^{sf_i} cos(\theta_i(s))ds \\ y_i(sf_i) &= \int_0^{sf_i} sin(\theta_i(s))ds \\ \theta_i(s) &= \int_0^{s} \kappa_i(s)ds \end{aligned}$$

C. Initial guess

The nonlinear solver is sensitive to initial guess, and hence an informed initial guess that satisfies all the constraints is critical. Because we use 3 curvature-polynomials for the 3 segments, we may use a Dubins path to seed each of the 3 segments. To do this, we take each curve and first calculate the change of heading that each curve provides along with the direction of curvature (R or L). Using this, we solve a least-squares optimiser that outputs an initial vector based on the Dubins solution. Care is taken that the initial vector satisfies curvature and curvature rate constraints while formulating the least-squares problem. This process is only used to produce seeds for a subset of the paths after which solutions in the library may act as seeds to generate rest of the library.

IV. BIT* Planner

This section details the domain-specific treatment of the BIT* path planning algorithm. BIT* is a class of tree-based

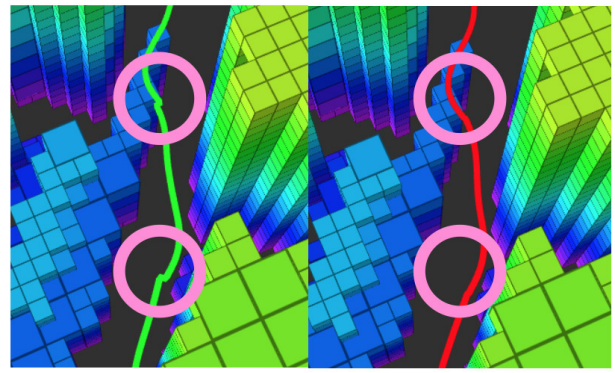


Fig. 4: Paths before (left) and after (right) repairing the path

incremental asymptotically optimal path planners. BIT* combines the use of heuristically ordered searches in a batch-wise fashion with the ability to focus the search on a sub-problem once the initial solution is available. These attributes significantly reduce the number of queries to find the actual cost function as opposed to RRT* making BIT* an attractive candidate for integration with BVPs [20], [21]. For a detailed explanation of the algorithm, we direct readers to the original paper [17].

A. Trajectory Library in lieu of BVP computations

As mentioned earlier, BIT* relies on the BVP solution to give exact connections between the states. While the performance of our proposed solution to the BVP is in 100s of milliseconds (see Section V), it is still not fast enough to be directly used to calculate cost functions and perform collision checks in real-time thus motivating the creation of a trajectory library containing pre-calculated primitives that can be used instead of solving the BVP. These solutions are modified in real-time using the wind conditions from the BIT* query. The trajectory library has precomputed solutions for a set of BVPs using a predetermined discretization of the state space. Without loss of generality, the start point of the BVP is kept at the origin, while the endpoint is varied inside the set bounds. Each solution containing the vector of optimization variable $\mathbf{p}$ is recorded along with the endpoint state. The BVPs are solved for a zero wind case. The level of discretization determines how close the actual solution will be from the query.

B. Algorithm Details

To successfully execute an instance of the planner, the planner needs the ability to query the cost and validity of a path between any two states $\{\mathbf{x}_s, \mathbf{x}_f\} \in \mathbb{C}_{free}$. For both these queries, it is important to find a primitive from the trajectory library that is spatially closest to the final path. A method to perform this search is captured in Algorithm 1. For every two-point query from the planner between states $\{\mathbf{x}_s, \mathbf{x}_f\}$ and the uniform wind $\mathbf{w}$ between them, we call the *GetPrimitive()* function. Using *FwdTransform()* [Line 2] the problem is first transformed into a frame of reference of the trajectory library. For this transformed problem, we iterate

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 *GetPrimitive( )*

1: **function** GETPRIMITIVE($\mathbf{x}_s, \mathbf{x}_f, \mathbf{w}$)
2: $[\mathbf{x_t}, \mathbf{w_t}] \leftarrow FwdTransform(\mathbf{x}_s, \mathbf{x}_f, \mathbf{w})$
3: $c \leftarrow \infty$
4: **for all** L $\in$ TrajLib **do** ▷ Loop over the library
5: $[\mathbf{x}_L, \mathbf{p}_L] \leftarrow L$ ▷ Extract end point and primtive
6: $\Delta\mathbf{x} \leftarrow \mathbf{x_L} - \mathbf{x_t}$ ▷ Spatial error
7: $sf \leftarrow |\mathbf{p}_L(end-3:end)|_1$ ▷ Path length
8: $c' \leftarrow |\Delta\mathbf{x} - \frac{\mathbf{w_t} sf}{V_a}|_2$ ▷ Wind-corrected error norm
9: **if** $c' \leq c$ **then**
10: $\mathbf{p} \leftarrow \mathbf{p}_L$
11: $c \leftarrow c'$
12: **end if**
13: **end for**
14: **return p**
15: **end function**

through the library [Line 4] to get the end point and the primitive for each member. The function then finds a member in the trajectory library [Line 5] which, when modified with the wind [Line 8], gives the spatially closest primitive to the transformed problem. The planner uses this surrogate solution to calculate the cost and perform collision checking.

The procedure for validity checking is detailed in Algorithm 2. To check if a path between two states is valid, i.e. $\in \mathbb{C}_{free}$, we first call *GetPrimitive()* [Line 2] to get the closest primitive. Then the spatial path is calculated from the primitive using *GetPath()* (refer Eq. 1 and 6). The path is then transformed back to the original frame using *BkdTransform()* [Line 4]. The function *CollisionAndGlideSlopeCheck()* [Line 5] validates if the states in the path are in free space, and also checks if the paths are within a glideslope threshold. When these checks pass, a path is returned as valid.

Once the planner returns a final path, we use the endpoints of each section to construct the BVP and use the corresponding primitives as seeds to the nonlinear constrained optimization problem detailed in Equation 7 to give out the final solution. The repair process for creating a smooth continuous path can be seen in Figure 4.

Algorithm 2 *MotionValidator( )*

1: **function** MOTIONVALIDATOR($\mathbf{x}_s, \mathbf{x}_f, \mathbf{w}$)
2: $\mathbf{q} \leftarrow GetPrimitive(\mathbf{x}_s, \mathbf{x}_f, \mathbf{w})$
3: $\Phi_t \leftarrow GetPath(\mathbf{q}, \mathbf{w})$
4: $\Phi \leftarrow BkdTransform(\Phi_t, \mathbf{x_s})$
5: **if** $CollisionAndGlideSlopeCheck(\Phi)$ **then**
6: **return** Φ
7: **else**
8: **return** NULL
9: **end if**
10: **end function**

V. IMPLEMENTATIONS AND FEASIBILITY STUDY

The BVP solver and global planner are implemented and tested in C++ to verify their real-time performance. In order to solve the nonlinear constrained optimization problem, we use COIN-OR's IPOPT solver [30]. We use the BIT* implementation in OMPL [31] to generate a plan for every query. A custom validity checker and cost generator is implemented. The trajectory library is generated for a $\pm 600m$ range on both x and y with 5x5m resolution and a 10° resolution on θ.

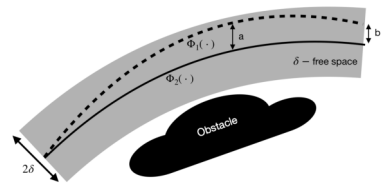


Fig. 5: Representation of ε-ratio between any two paths. Figure also shows that path $\Phi_1(\cdot)$ is δ-close to path $\Phi_2(\cdot)$

In order to use the primitives as surrogates to the BVP solutions for the planner, we need to characterize how closely the primitives represent the actual solution. To do this, we find empirical bounds on metrics that quantify the difference between primitive and the corresponding repaired BVP solution.

A. Cost Metric

The cost metric is the percentage change in the cost between the seed and the solution. This metric quantifies how far from the actual cost is the surrogate cost used by the planner. In order to find this metric bound, we ran the BVP solver for 10,000 randomly sampled points within the bounds of the trajectory library with a varying wind of unit magnitude. The path lengths before and after repair were compared, and the percentage change is plotted in Figure 7 with a 95% confidence interval.

B. ε- metric

The ε- metric measures how much the path can spatially move after repairing. In order to define this metric, we need the following definitions.

Definition 1: δ-clear state
Any state $\mathbf{x} \in \mathbb{C}_{free}$ is said to be a δ-clear state if all the states within a δ euclidean distance from $\mathbf{x}$ are also in $\mathbb{C}_{free}$

Definition 2: δ-clear path
Any path $\Phi(\cdot)$ is said to be δ-clear path if all the states $\mathbf{x} \in \Phi(\cdot)$ are δ-clear states

Definition 3: δ-clear space
Any space within a δ euclidean distance of a δ-clear path $\Phi(\cdot)$ is said to be the δ-clear space of that $\Phi(\cdot)$ path.

Definition 4: δ-close
Given two paths $\Phi_1(\cdot)$ and $\Phi_2(\cdot)$, $\Phi_1(\cdot)$ is said to be δ-close to $\Phi_1(\cdot)$ if $\forall \mathbf{x} \in \Phi_1(\cdot)$, $\mathbf{x}$ are in δ-clear space of $\Phi_2(\cdot)$.

Definition 5: ε-ratio
Given two paths $\Phi_1(\cdot)$ and $\Phi_2(\cdot)$ with the same start states and path parametrization $t \in [0, \tau]$, an ε-ratio is defined:

$$\varepsilon = \frac{max_{t\in[0,\tau]}(||\Phi_1(t) - \Phi_2(t)||)}{||\Phi_1(\tau) - \Phi_2(\tau)||}$$

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

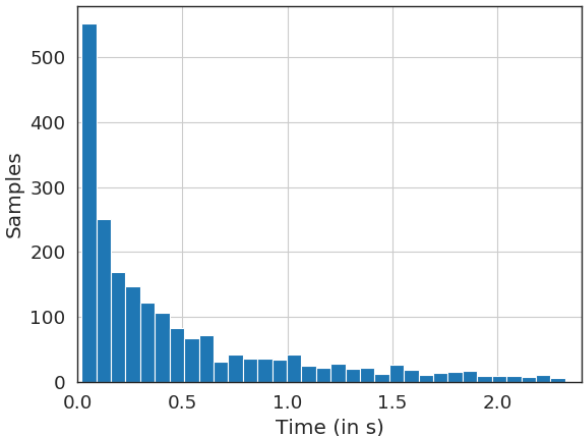


Fig. 6: Histogram of time taken to solve the BVP based on 3000 samples drawn randomly from the state space.

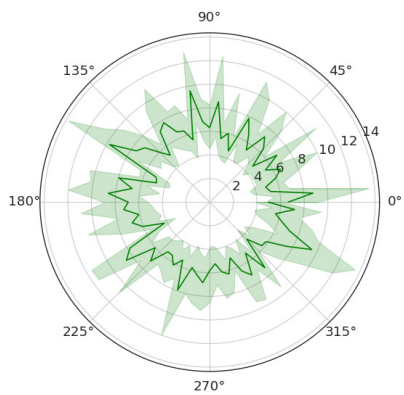


Fig. 7: The percentage change in cost between the surrogate solution from the library and its corresponding repaired solution. Angles show the direction of incoming wind. As seen, the change hovers around a mean of 6.5%.

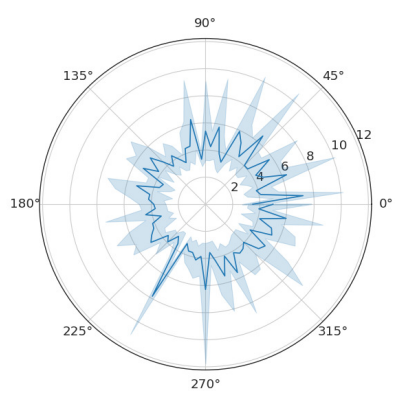


Fig. 8: ε-ratio between the surrogate solution from the library and its corresponding repaired solution. Angles show the direction of incoming wind.

Here we carry out a change of parametrization from s to t using $t(s) = \frac{s\tau}{sf}$ for both the paths.

Intuitively speaking, if we can put an empirical bound on the value of ε-ratio, then we can put a bound on how much a path is likely to diverge after it is repaired. This bound is a function of how far the endpoint of the surrogate is from the actual boundary condition, i.e. a solution is less likely to deviate from the surrogate if the boundary condition is almost satisfied. If the repaired path is δ-close to the surrogate solution, we can guarantee that the repaired path does not collide with any obstacles.
Consider $\Phi_2(\cdot)$ in Figure 5 to be the selected surrogate solution and $\Phi_1(\cdot)$ be the corrected one. As the value of b is known, we may choose a δ inflation factor for the obstacles such that $\delta > \varepsilon b$. For such a choice of δ, any repaired path will not collide with obstacles, and a collision check on the surrogate is equivalent to a collision check for the repaired path. In practice, getting a strict bound on the ε-ratio might be difficult, so we approximate the value by running multiple simulations and recording the value. In order to find the value, we ran the BVP solver in the same manner as for calculating the path length changes. The resulting value with a unit-magnitude wind direction and a 95% confidence interval is represented as radial plots in Figure 8.

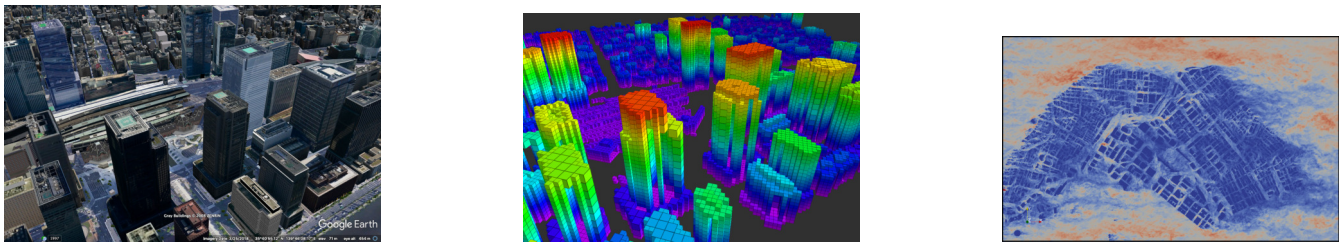
C. Computation Time

Analysis of computation time required to find and repair a primitive is essential to prove the feasibility of the planner. Figure 6 shows a histogram of the time required to solve BVPs for 3000 randomly sampled points in the state space. A varying wind with unit magnitude is also applied. The mean time to solve a BVP is 484.6 milliseconds. The primitive lookup for the precomputed trajectory library (Algorithm 1) can be slow if we iterate through the full library. One way to reduce this computation is to restrict the search in the trajectory library to an area which when transformed with wind is most likely to yield an optimal solution. Using this method we could achieve a lookup function that has a mean computation time of 0.256 milliseconds which is 3 orders of magnitude faster than the BVP computation time. All computations are performed on an Intel NUC8i5BEK with a processor base frequency of 2.30 GHz.

VI. Numerical Results

In order to validate the efficacy of the proposed algorithms, we use a 3D model of an actual city block in a major metropolitan city. The city block is 3300 $\times$ 2130 meters and consists of several high rise structures. The maximum and minimum flight altitudes are restricted to 100m and 20m, respectively. Figures 9a and 9b show the actual city and its Octomap [32] representation with the same view. The 3D model of the city block is fed into a comprehensive high-fidelity CFD simulator to generate a temporally-static but spatially non-uniform wind field. The wind enters the city from the positive X direction with unit magnitude. The

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.



(a) 3D map of a city block

(b) Octomap representation of the city block used for planning

(c) Wind Field in the city

Fig. 9: Urban environment used for simulation results

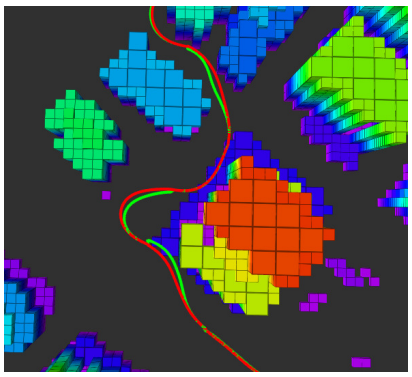


Fig. 10: Figure shows a section of the calculated path in the simulation environment before (green) and after (red) repair

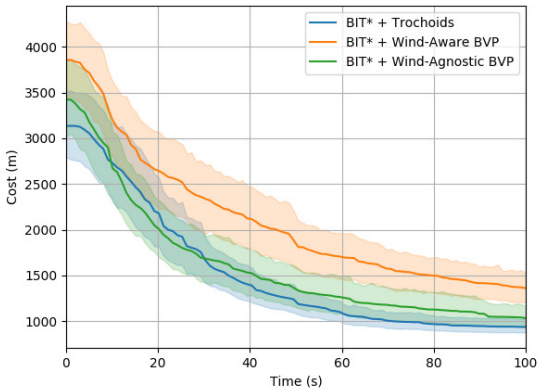


Fig. 11: Figure shows the simulation results with the wind-aware and wind-agnostic BVP solver. Comparative results with trochoids show the competitive performance of our approach.

interactions between the wind and structures form interesting wind patterns, as shown in Figure 9c.

To verify the convergence properties of the planner, path planning queries are generated to construct trajectories from one location of the environment to the other. In order to compare the performance of the algorithm, we also present results using trochoids as a connector in the BIT*. Figure 10 shows results from one run. A run is successful if the algorithm can find a path using surrogate solutions, can repair all the segments in the path and the final repaired path is collision-free. The repairing routine is run with multiple initializations to improve BVP performance. Wind agnostic results are also shown for completeness. Two types of benchmark tests are performed: For the first set of runs, a start and goal location was selected, and the algorithms were tested with 100 different pseudo-random seeds for each. The trochoids based planner could successfully find a valid trajectory 98% of the time, while our proposed algorithm could find a feasible solution with a success rate of 96% for wind-agnostic and 84% for wind-aware cases. The path costs vs time for the test runs with 95% confidence bounds are shown in Figure 11. As trochoids can take curvature discontinuous turns, they generate lower-cost solutions. Our approach provides curvature continuous paths that are longer because they are smoother and hence more feasible. Between wind-aware and wind-agnostic case, the costs are determined by the underlying flow field and in this specific case, the wind-aware costs are higher. While our approach does pay a penalty in success rate, the final costs and convergence rates are competitive against even curvature discontinuous trochoid based connectors. To further verify the performance of the planner, 100 sets of two points were randomly selected from $\mathbb{C}_{free}$ space, and path planning queries were generated. 93% of the queries were able to produce trajectories that were fully repaired and did not result in a collision.

VII. Future Work

While the results are encouraging, more can be done to improve the robustness of the proposed methodology. The authors could identify two drawbacks. Firstly, while the use of surrogate solutions improves the real-time performance of the sampling-based planner, it leads to a final sub-optimal solution. Secondly, there is no guarantee provided in the strategy that the chosen primitive will lead to convergence when attempting to repair the generated plan. The authors found that convergence rates for repairs are unfavourably affected by higher wind magnitudes. While a straightforward workaround is to try with multiple primitives while repairing, a more structured and possibly predictive approach can improve the system performance. Non-convergence may also result from the BVP trying to find solutions to dynamically

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.

unreachable states, as the planner has no information about the forward reachability of the state it is trying to expand.

Most of these issues can be addressed by incorporating a more comprehensive strategy for choosing primitives from the trajectory library. One possible approach is to assign a probability distribution on the trajectory library conditioned on the query at hand. Such a distribution can encode how likely is a particular primitive to converge on a given query.

VIII. Conclusions

A novel method for producing collision-free dynamically feasible trajectories in a known wind field is presented. To the best of authors' knowledge, this is the only real-time near-optimal BVP-based implementation for curvature continuous wind-aware path planning. The proposal uses a library of precomputed two-point boundary value problem solutions as connectors in sampling-based planners. The final path was produced by repairing the surrogate solutions to produce a smooth continuous path. Benchmark tests in a realistic simulation environment are performed, and comparative results with a trochoid-based BVP planner are used to showcase the performance of the proposed strategy. The algorithm was able to produce wind-aware curvature-continuous paths in competitive time frames and cost ranges.

References

[1] D. Maturana, S. Arora, and S. Scherer, "Looking forward: A semantic mapping system for scouting with micro-aerial vehicles," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 6691–6698.

[2] J. K. Stolaroff, C. Samaras, E. R. O'Neill, A. Lubers, A. S. Mitchell, and D. Ceperley, "Energy use and life cycle greenhouse gas emissions of drones for commercial package delivery," *Nature communications*, vol. 9, no. 1, p. 409, 2018.

[3] D. Gallacher, "Drone applications for environmental management in urban spaces: A review," *International Journal of Sustainable Land Use and Urban Planning*, vol. 3, no. 4, 2016.

[4] B. Z. Cybyk, B. E. McGrath, T. M. Frey, D. G. Drewry, J. F. Keane, and G. Patnaik, "Unsteady airflows and their impact on small unmanned air systems in urban environments," *Journal of Aerospace Information Systems*, vol. 11, no. 4, pp. 178–194, 2014.

[5] C. Liu, O. McAree, and W.-H. Chen, "Path following for small uavs in the presence of wind disturbance," in *Proceedings of 2012 UKACC International Conference on Control*. IEEE, 2012, pp. 613–618.

[6] A. Ailon and I. Zohar, "Control of unmanned aerial vehicle with restricted input in the presence of additive wind perturbations," in *2011 11th International Conference on Control, Automation and Systems*. IEEE, 2011, pp. 873–878.

[7] J. Patrikar, V. R. Makkapati, A. Pattanaik, H. Parwana, and M. Kothari, "Nested saturation based guidance law for unmanned aerial vehicles," *Journal of Dynamic Systems, Measurement, and Control*, vol. 141, no. 7, p. 071008, 2019.

[8] S. L. Herbert, M. Chen, S. Han, S. Bansal, J. F. Fisac, and C. J. Tomlin, "Fastrack: a modular framework for fast and guaranteed safe motion planning," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 1517–1522.

[9] V. Stepanyan and K. S. Krishnakumar, "Estimation, navigation and control of multi-rotor drones in an urban wind field," in *AIAA Information Systems-AIAA Infotech@ Aerospace*, 2017, p. 0670.

[10] D. N. Subramani, Q. J. Wei, and P. F. Lermusiaux, "Stochastic time-optimal path-planning in uncertain, strong, and dynamic flows," *Computer Methods in Applied Mechanics and Engineering*, vol. 333, pp. 218–237, 2018.

[11] I. Mir, S. A. Eisa, and A. Maqsood, "Review of dynamic soaring: technical aspects, nonlinear modeling perspectives and future directions," *Nonlinear Dynamics*, vol. 94, no. 4, pp. 3117–3144, 2018.

[12] N. R. Lawrance and S. Sukkarieh, "Path planning for autonomous soaring flight in dynamic wind fields," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 2499–2505.

[13] A. Bahabry, X. Wan, H. Ghazzai, H. Menouar, G. Vesonder, and Y. Massoud, "Low-altitude navigation for multi-rotor drones in urban areas," *IEEE Access*, vol. 7, pp. 87 716–87 731, 2019.

[14] S. Primatesta, G. Guglieri, and A. Rizzo, "A risk-aware path planning strategy for uavs in urban environments," *Journal of Intelligent & Robotic Systems*, vol. 95, no. 2, pp. 629–643, 2019.

[15] J. Ware and N. Roy, "An analysis of wind field estimation and exploitation for quadrotor flight in the urban canopy layer," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 1507–1514.

[16] B. Nurimbetov, O. Adiyatov, S. Yeleu, and H. A. Varol, "Motion planning for hybrid uavs in dense urban environments," in *2017 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*. IEEE, 2017, pp. 1627–1632.

[17] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 3067–3074.

[18] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller, "Anytime motion planning using the rrt," in *2011 IEEE International Conference on Robotics and Automation*. IEEE, 2011, pp. 1478–1483.

[19] R. E. Allen and M. Pavone, "A real-time framework for kinodynamic planning in dynamic environments with application to quadrotor obstacle avoidance," *Robotics and Autonomous Systems*, vol. 115, pp. 174–193, 2019.

[20] C. Xie, J. van den Berg, S. Patil, and P. Abbeel, "Toward asymptotically optimal motion planning for kinodynamic systems using a two-point boundary value problem solver," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 4187–4194.

[21] M. Lan, S. Lai, Y. Bi, H. Qin, J. Li, F. Lin, and B. M. Chen, "Bit*-based path planning for micro aerial vehicles," in *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2016, pp. 6079–6084.

[22] B. Sakcak, L. Bascetta, G. Ferretti, and M. Prandini, "Sampling-based optimal kinodynamic planning with motion primitives," *Autonomous Robots*, pp. 1–18, 2019.

[23] Y. Li, Z. Littlefield, and K. E. Bekris, "Sparse methods for efficient asymptotically optimal kinodynamic planning," in *Algorithmic foundations of robotics XI*. Springer, 2015, pp. 263–282.

[24] V. Dugar, S. Choudhury, and S. Scherer, "A κite in the wind: Smooth trajectory optimization in a moving reference frame," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 109–116.

[25] L. Techy and C. A. Woolsey, "Minimum-time path planning for unmanned aerial vehicles in steady uniform winds," *Journal of guidance, control, and dynamics*, vol. 32, no. 6, pp. 1736–1746, 2009.

[26] S. Schopferer, J. S. Lorenz, A. Keipour, and S. Scherer, "Path planning for unmanned fixed-wing aircraft in uncertain wind conditions using trochoids," in *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2018, pp. 503–512.

[27] S. Schopferer and T. Pfeifer, "Performance-aware flight path planning for unmanned aircraft in uniform wind fields," in *2015 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2015, pp. 1138–1147.

[28] A. Kelly and B. Nagy, "Reactive nonholonomic trajectory generation via parametric optimal control," *The International Journal of Robotics Research*, vol. 22, no. 7-8, pp. 583–601, 2003.

[29] Y. Lin and S. Saripalli, "Path planning using 3d dubins curve for unmanned aerial vehicles," in *2014 international conference on unmanned aircraft systems (ICUAS)*. IEEE, 2014, pp. 296–304.

[30] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.

[31] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, http://ompl.kavrakilab.org.

[32] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, no. 3, pp. 189–206, 2013.

Authorized licensed use limited to: University of New South Wales. Downloaded on October 18,2020 at 12:06:06 UTC from IEEE Xplore. Restrictions apply.